

SQL (Structured Query Language Execution)

Milenko Burlica

28 July, 2026

Table of Contents

1	 Automatische Datenbank-Konfiguration (Connection Handling)	4
---	--	---

Die SQL-Engine dient zur Ausführung von Datenbankbefehlen direkt auf den konfigurierten Datenbanken des Frameworks. Sie reagiert dediziert auf den Testschritt-Typ `type: SQL`.

💡 Das Entkopplungs-Prinzip (Separation of Concerns) Die Engine trennt die Ausführung von der Validierung strikt:

- Ein Befehl wird einmalig ausgeführt (`op: EXEC`) und das tabellarische Ergebnis (`List<Map<String, Object>>`) unter einem eindeutigen Namen (`response`) im Execution-Context zwischengespeichert.
- Anschließend können beliebig viele Prüfungen (`op: ASSERT`) oder Datenextraktionen (`op: BUFFER`) auf diesem gespeicherten Ergebnis durchgeführt werden, ohne die Datenbank erneut zu belasten.

1 Automatische Datenbank-Konfiguration (Connection Handling)

Die SQL-Engine arbeitet vollkommen transparent und benötigt im YAML-Testschritt **keine** Angaben zu Passwörtern, Usern oder JDBC-URLs.

Die Verbindungsinformationen werden beim Start des Frameworks automatisch und zentral über den `EnvironmentManager` aus der globalen Konfigurationsdatei der aktuellen Testumgebung (`env/` `env.config`) geladen.

Die Engine liest dabei dediziert folgende Keys aus:

- `database.connectionString` – Die JDBC-Verbindungs-URL zur Ziel-Datenbank (z.B. Oracle, PostgreSQL, etc.).
- `database.user` – Der Datenbank-Benutzer.
- `database.password` – Das (verschlüsselte) Passwort für den Datenbankzugriff.

Globales Feature: `condition` (Bedingte Ausführung) Mit dem optionalen Feld `condition` kann gesteuert werden, ob ein SQL-Testschritt ausgeführt werden soll. Wenn die Bedingung nicht zutrifft (`false`), wird der Schritt übersprungen (**SKIPPED**) und der Test läuft fehlerfrei weiter.

YAML

```
# Führt den Befehl nur aus, wenn die TestCase-Variable "run_db_setup" auf true steht
- type: SQL
  condition: '{B[run_db_setup]}' == 'true'
  op: EXEC
  command: "SELECT * FROM SYS_PARAMETER"
  response: param_table
```

Engine-spezifische Parameter Neben den Standardfeldern besitzt die SQL-Engine spezielle Steuerungs-Parameter im YAML:

- **command** (String, Pflichtfeld bei `op: EXEC`): Das auszuführende SQL-Statement (`SELECT` , `UPDATE` , `INSERT` , `DELETE`). Unterstützt mehrzeilige Notationen mittels des Pipe-Zeichens (`|`).
- **response** (String, Pflichtfeld bei `op: EXEC`): Der eindeutige Schlüssel, unter dem das Result-Set im Buffer für nachfolgende `ASSERT` - oder `BUFFER` -Schritte hinterlegt wird.
- **row** (Integer/String, Optional): Die Zeilennummer des Result-Sets (beginnend bei `0`). Wird bei `BUFFER` und `ASSERT` verwendet. Fehlt die Angabe, wird standardmäßig Zeile `0` herangezogen.

- **column** (String): Der Name der Tabellenspalte (Case-Insensitive) oder der numerische Spaltenindex (z.B. `0` für die erste Spalte), auf die zugegriffen werden soll.

Unterstützte Operationen (op)

1. Operation: EXEC (Kommando auf DB ausführen) Führt ein SQL-Statement direkt über den internen `DatabaseManager` aus. Die Engine unterscheidet automatisch anhand des Statements:
 - **SELECT-Statements:** Holt die Daten ab, speichert sie als Tabelle im Context und gibt das Resultat formatiert in der Konsole aus.
 - **DML-Statements (UPDATE/INSERT/DELETE):** Führt die Änderung auf der Datenbank aus und liefert die Anzahl der betroffenen Zeilen zurück (`Affected Rows`).

YAML

```
- type: SQL
  op: EXEC
  command: |
    SELECT COUNT(*) as PARAM_COUNT
    FROM SYS_PARAMETER
    WHERE STATUS = 'ACTIVE'
  response: response_sql1
```

1. Operation: ASSERT (Validierung) Prüft den Inhalt oder die Struktur eines zuvor gespeicherten SQL-Ergebnisses (`response`). Schlägt eine Prüfung fehl, bricht der gesamte Test sofort ab (Fail-Fast).

Aktion (action)	Beschreibung	Verwendung im YAML
EQUALS / NOT_EQUALS	Prüft den Zelleninhalt auf exakte Gleichheit oder Ungleichheit.	<code>expected: "2454"</code>
CONTAINS	Prüft, ob der Zelleninhalt den Suchtext enthält.	<code>expected:</code> <code>"Active"</code>
ROW_COUNT	Überprüft die Gesamtanzahl der Zeilen im Result-Set.	<code>expected: "1"</code>
ALL_MATCH / ANY_MATCH	Kollektionsprüfung: Trifft auf alle bzw. mindestens eine Zeile der Spalte zu.	<code>expected:</code> <code>"COMPLETED"</code>

Aktion (action)	Beschreibung	Verwendung im YAML
BETWEEN	Prüft, ob ein numerischer Zellenwert innerhalb einer Range liegt.	<code>expected:</code> <code>"100..200"</code>
DATE_EQUALS	Erlaubt smarte Datumsvergleiche (Unterstützt Schlüsselwörter wie <code>TODAY</code> , <code>YESTERDAY</code>).	<code>expected:</code> <code>"TODAY"</code>
IS_NULL / IS_NOT_NULL	Überprüft das Feld auf den Datenbank-Wert <code>null</code> .	Spalte definieren
IS_EMPTY / IS_NOT_EMPTY	Prüft, ob das Feld <code>null</code> , <code>"null"</code> oder ein Leerstring ist.	Spalte definieren

YAML

```
# Inhaltsprüfung auf dem zuvor gespeicherten "response_sql1" Output
- type: SQL
  op: ASSERT
  row: 0
  column: "PARAM_COUNT"
  action: EQUALS
  expected: "2454"
  response: response_sql1
```

1. Operation: BUFFER (Datenextraktion) Liest gezielt den Wert einer bestimmten Zelle (`row` und `column`) aus einer gespeicherten SQL-Response und legt sie als neue Variable (`name`) im TestCase ab, damit nachfolgende Schritte (auch für REST, OC oder CMD) darauf zugreifen können.

YAML

```
# Holt den Wert aus Zeile 0 der Spalte "PARAM_COUNT" aus dem Buffer "response_sql1"
- type: SQL
  op: BUFFER
  row: 0
  column: "PARAM_COUNT"
  name: targetVar
  response: response_sql1
```

 **Konsole-Logging & Live-Ausgabe (Log Alignment)** Die SQL-Engine verfügt über ein strikt vertikal ausgerichtetes Logging-System, das perfekt auf das einheitliche Framework-Format (4 Spaces Einrückung) abgestimmt ist.

Plaintext

```
>>> SQL EXEC      :
                        SELECT
                        CURRENT_DATE as TODAY_COL,
                        150.55 as PRICE,
                        NULL as DELETED_AT,
                        'Active' as STATUS
                        FROM DUAL

<<< +-----+-----+-----+-----+
<<< | TODAY_COL          | PRICE | DELETED_AT | STATUS |
<<< +-----+-----+-----+-----+
<<< | 31.05.2026 00:22:03 | 150.55 | null       | Active |
<<< +-----+-----+-----+-----+
<<< [Total Rows: 1]
>>> RESULT          : SUCCESS (Rows: 1)

>>> ASSERT          : [TODAY_COL, Row: 0] DATE_EQUALS "TODAY" (Ref: res_demo)
>>> RESULT          : SUCCESS (Value is "31.05.2026 00:22:03")

>>> BUFFER          : Store SQL cell [PRICE, Row: 0] to variable [buffered_price]
<<< OUT              : 150.55
>>> RESULT          : SUCCESS (Stored)
```

Wichtige YAML-Syntaxregeln & Datums-Handling

1. Schutz von Dynamic-Tags { } Wenn ein Feld (wie z.B. `expected` oder `command`) mit einem Platzhalter beginnt, muss der gesamte Wert in Anführungszeichen gesetzt werden, da die geschweiften Klammern sonst den YAML-Parser blockieren.
 -  Falsch: `expected: {B[my_variable]}`
 -  Richtig: `expected: "{B[my_variable]}"`
1. Smarte Datums- und Numerik-Präfixe Die Engine unterstützt dynamische Typerkennungen innerhalb des `expected`-Feldes bei Assertions:
 - **NUMBER: -Präfix:** Wandelt den erwarteten Wert explizit in eine Zahl für mathematische Vergleiche um (z.B. `expected: "NUMBER:10"` bei `GREATER_THAN`).
 - **DATE: -Präfix:** Ermöglicht Berechnungen direkt im YAML. Ausdrücke wie `DATE:SYSDATE-10` oder `DATE:SYSDATE+2` werden automatisch in das Standard-Datumsformat (`yyyy-MM-dd`) aufgelöst und mit dem Datenbankwert abgeglichen.

1. Mehrzeilige SQL-Statements (Block-Notation) Für komplexe, lange `SELECT`-Statements oder Joins sollte immer die YAML-Block-Notation mittels Pipe (`|`) verwendet werden. Das erhöht die Lesbarkeit im Code und verhindert Fehler beim Parsing von Anführungszeichen.